

Assignment 4: Data Wrangling

Ashlee Warren

OVERVIEW

In this assignment, you will build a reproducible workflow that cleans four EPA air-quality files, combines them, reshapes the result, and produces grouped summaries. Date conversion is a brief review from Module 3; the emphasis is on data wrangling.

Directions

1. Rename this file `<FirstLast>_A04_DataWrangling.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. Ensure that code in code chunks does not extend off the page in the PDF.

Set up your session

Questions 1–2

- 1a. Load the `tidyverse` and `here` packages into your session.
 - 1b. Check your working directory.
 - 1c. Read all four raw files associated with the EPA Air dataset. See the README file for information about the variables and measurements.
2. Add the appropriate code to reveal the dimensions of the four datasets.

```
#1a Loading the tidyverse and here packages
```

```
library(tidyverse)
library(here)
```

```
#1b Checking the location of my working directory
```

```
here()
```

```
## [1] "/home/guest/EDE_R_Lab_F26_NEW"
```

```
#1c Adding the four raw data files for the EPA Air dataset
```

```
EPAair.03.NC2024 <- read.csv(
  here("Data/Raw/EPAair_03_NC2024_raw.csv")
)
```

```

EPAair.O3.NC2025 <- read.csv(
  here("Data/Raw/EPAair_O3_NC2025_raw.csv")
)

EPAair.PM25.NC2024 <- read.csv(
  here("Data/Raw/EPAair_PM25_NC2024_raw.csv")
)

EPAair.PM25.NC2025 <- read.csv(
  here("Data/Raw/EPAair_PM25_NC2025_raw.csv")
)

#I am also reviewing the metadata for each to understand the datasets and variables.

#2 #Adding code to find the dimensions of each dataset
dim(EPAair.O3.NC2024)

## [1] 9519    21

#EPAair.O3.NC2024 has 9,519 observations of 21 variables

dim(EPAair.O3.NC2025)

## [1] 9466    21

#EPAair.O3.NC2025 has 9,466 observations of 21 variables

dim(EPAair.PM25.NC2024)

## [1] 9752    22

#EPAair.PM25.NC2024 has 9,752 observations of 22 variables

dim(EPAair.PM25.NC2025)

## [1] 10184    22

#EPAair.PM25.NC2025 has 10,184 observations of 22 variables

```

All four datasets should have the same number of columns (PM2.5 has one extra column), but unique record counts (rows). Do your datasets follow this pattern? Yes, my O3 datasets both have 21 columns, and my PM25 datasets have 22 columns. Each of the four datasets have unique number of rows.

Wrangle individual datasets to create processed files.

Questions 3–5

3. Change the Date columns to be date objects.
4. Select the following columns: DDate, Daily.AQI.Value, Local.Site.Name, AQS.Parameter.Description, County, Site.Latitude, Site.Longitude
5. For the PM2.5 datasets, fill all cells in AQS.Parameter.Description with “PM2.5” (all cells in this column should be identical).

#3 Changing the date columns in all four datasets from character objects to date objects

```
EPAair.03.NC2024$Date <- mdy(EPAair.03.NC2024$Date)
```

```
EPAair.03.NC2025$Date <- mdy(EPAair.03.NC2025$Date)
```

```
EPAair.PM25.NC2024$Date <- mdy(EPAair.PM25.NC2024$Date)
```

```
EPAair.PM25.NC2025$Date <- mdy(EPAair.PM25.NC2025$Date)
```

*#4 Selecting these columns from all four datasets: Date, Daily.AQI.Value,
#Local.Site.Name, AQS.Parameter.Description, County, Site.Latitude,
#Site.Longitude*

```
EPAair.03.NC2024.processed <- select(EPAair.03.NC2024, Date, Daily.AQI.Value,  
                                   Local.Site.Name, AQS.Parameter.Description,  
                                   County:Site.Longitude)
```

```
EPAair.03.NC2025.processed <- select(EPAair.03.NC2025, Date, Daily.AQI.Value,  
                                   Local.Site.Name, AQS.Parameter.Description,  
                                   County:Site.Longitude)
```

```
EPAair.PM25.NC2024.columnssubset <- select(EPAair.PM25.NC2024, Date, Daily.AQI.Value,  
                                           Local.Site.Name, AQS.Parameter.Description,  
                                           County:Site.Longitude)
```

```
EPAair.PM25.NC2025.columnssubset <- select(EPAair.PM25.NC2025, Date, Daily.AQI.Value,  
                                           Local.Site.Name, AQS.Parameter.Description,  
                                           County:Site.Longitude)
```

*#5 First, I want to understand what is currently in AQS.Parameter.Description column
#of the PM25 datasets, so I am viewing the 2024 dataset and then commenting it out.
#Next, I'm changing all cells in the AQS.Parameter.Description column to "PM2.5". I
#am then viewing both datasets to ensure that it worked and commenting out view.*

```
#view(EPAair.PM25.NC2024.columnssubset)
```

```
EPAair.PM25.NC2024.processed <- mutate(EPAair.PM25.NC2024.columnssubset,  
                                       AQS.Parameter.Description = "PM2.5")
```

```
#view(EPAair.PM25.NC2024.processed)
```

```
EPAair.PM25.NC2025.processed <- mutate(EPAair.PM25.NC2025.columnsubset,
                                         AQS.Parameter.Description = "PM2.5")

#view(EPAair.PM25.NC2025.processed)
```

Question 6

6. Save all four processed datasets in the Processed folder. Use the same file names as the raw files but replace “raw” with “processed”.

#6 Saving all four processed datasets in the Processed folder and replacing "raw" with "processed" in the file name.

```
write.csv(
  EPAair.03.NC2024.processed,
  row.names = FALSE,
  file = here("Data/Processed/EPAair_03_NC2024_processed.csv")
)

write.csv(
  EPAair.03.NC2025.processed,
  row.names = FALSE,
  file = here("Data/Processed/EPAair_03_NC2025_processed.csv")
)

write.csv(
  EPAair.PM25.NC2024.processed,
  row.names = FALSE,
  file = here("Data/Processed/EPAair_PM25_NC2024_processed.csv")
)

write.csv(
  EPAair.PM25.NC2025.processed,
  row.names = FALSE,
  file = here("Data/Processed/EPAair_PM25_NC2025_processed.csv")
)
```

Combine datasets

Question 7

7. Stack the four datasets with `bind_rows()`. Check that their column names are identical first.

#7 Checking the column names for each dataset

```
colnames(EPAair.03.NC2024.processed)
```

```
## [1] "Date"                "Daily.AQI.Value"
## [3] "Local.Site.Name"     "AQS.Parameter.Description"
## [5] "County"              "Site.Latitude"
## [7] "Site.Longitude"
```

```
colnames(EPAair.03.NC2025.processed)
```

```
## [1] "Date" "Daily.AQI.Value"
## [3] "Local.Site.Name" "AQS.Parameter.Description"
## [5] "County" "Site.Latitude"
## [7] "Site.Longitude"
```

```
colnames(EPAair.PM25.NC2024.processed)
```

```
## [1] "Date" "Daily.AQI.Value"
## [3] "Local.Site.Name" "AQS.Parameter.Description"
## [5] "County" "Site.Latitude"
## [7] "Site.Longitude"
```

```
colnames(EPAair.PM25.NC2025.processed)
```

```
## [1] "Date" "Daily.AQI.Value"
## [3] "Local.Site.Name" "AQS.Parameter.Description"
## [5] "County" "Site.Latitude"
## [7] "Site.Longitude"
```

```
#Confirmed that the column names are the same and am binding the rows together into on dataframe
EPAair.03.PM25.2024.2025 <- bind_rows(EPAair.03.NC2024.processed,
                                     EPAair.03.NC2025.processed,
                                     EPAair.PM25.NC2024.processed,
                                     EPAair.PM25.NC2025.processed)
```

Question 8

8. Use one pipeline (%>%) to create a daily site-level dataset that meets the following conditions:

- Include only sites that the four data frames have in common:

“Linville Falls”, “Durham Armory”, “Leggett”, “Hattie Avenue”,
“Clemmons Middle”, “Mendenhall School”, “Frying Pan Mountain”, “West Johnston Co.”, “Garinger High School”, “Castle Hayne”, “Pitt Agri. Center”, “Bryson City”, “Millbrook School”

(the function `intersect` can figure out common factor levels - but it will include sites with missing site information, which you don’t want...)

- Some sites have multiple measurements per day. Group by date, site name, AQS parameter, and county, then calculate the mean AQI value, latitude, and longitude. Handle missing values explicitly in each mean.
- Add numeric `Month` and `Year` columns from `Date`. Check if results are integers.
- Hint: the dimensions of this dataset should be $15,012 \times 9$.

#8 I am using a pipe to create a daily site-level dataset from my combined dataframe. #First, I am using the filter argument to filter the Local Site Name column to only #include the sites that all four datasets have in common. Second, I am using the #group_by function to group by date, site name, AQS parameter, and county. Next, I #am using the summarize function to create new columns that have the mean for the #Daily AQI value, the Site Latitude, and the Site Longitude and preserving the NAs #while asking the mean to ignore them. I'm ending that function with .groups = "drop" #to remove the grouping after the summary is complete. Next, I am using the mutate #argument to add columns for the month and year and using as.integer to ensure that #they are stored as integers rather than numerics.

```
EPAair.03.PM25.2024.2025.DailySiteLevel <-
  EPAair.03.PM25.2024.2025 %>%
  filter(Local.Site.Name %in% c("Linville Falls", "Durham Armory", "Leggett",
    "Hattie Avenue", "Clemmons Middle", "Mendenhall School",
    "Frying Pan Mountain", "West Johnston Co.",
    "Garinger High School", "Castle Hayne", "Pitt Agri. Center",
    "Bryson City", "Millbrook School")) %>%
  group_by(Date, Local.Site.Name, AQS.Parameter.Description, County) %>%
  summarize(
    Mean.Daily.AQI = mean(Daily.AQI.Value, na.rm = TRUE),
    Mean.Site.Latitude = mean(Site.Latitude, na.rm = TRUE),
    Mean.Site.Longitude = mean(Site.Longitude, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(Month = as.integer(month(Date)),
    Year = as.integer(year(Date)))
```

Questions 9–10

9. Use `pivot_wider()` so that ozone and PM2.5 AQI values occupy separate columns. Each location-date combination should occupy one row.
10. Call up the dimensions of your new tidy dataset.

#9 I'm using the pivot_wider so that ozone and PM2.5 values have separate columns.

```
EPAair.03.PM25.2024.2025.DailySiteLevel.wide <-
  EPAair.03.PM25.2024.2025.DailySiteLevel %>%
  pivot_wider(
    names_from = AQS.Parameter.Description,
    values_from = Mean.Daily.AQI
  )
```

#view(EPAair.03.PM25.2024.2025.DailySiteLevel.wide) checked to make sure the new dataframe #looked as intended and am not commenting it out.

#10 Using dim to get the dimensions of my wide dataframe.

```
dim(EPAair.03.PM25.2024.2025.DailySiteLevel.wide)
```

```
## [1] 9037    9
```

#The wide dataframe has 9,037 observations of 9 variables.

Question 11

11. Save your processed dataset with the following file name: “EPAair_O3_PM25_NC2425_Processed.csv”

#11 Saving the wide dataframe as a new processed csv file

```
write.csv(  
  EPAair.O3.PM25.2024.2025.DailySiteLevel.wide,  
  row.names = FALSE,  
  file = here("Data/Processed/EPAair_O3_PM25_NC2425_Processed")  
)
```

Generate summary tables

Questions 12–13

12. Create a monthly summary data frame grouped by site, month, and year. Calculate mean ozone and PM2.5 AQI for each group, handling missing values explicitly. Then use `drop_na()` to remove rows only when mean **ozone** is missing. Mean PM2.5 may remain missing.
13. Call up the dimensions of the summary dataset.

#12 Creating a monthly summary dataframe. First, I'm using group_by to group by the site, #month, and year. Next, I'm calculating the mean O3 and PM2.5, ignoring the NAs, and #removing the groupings after the summary is complete. Finally, I'm removing the rows #with mean O3 is missing.

```
EPAair.O3.PM25.NC2425.MonthlySummary <-  
  EPAair.O3.PM25.2024.2025.DailySiteLevel.wide %>%  
  group_by(Local.Site.Name, Month, Year) %>%  
  summarize(mean.PM2.5 = mean(PM2.5, na.rm = TRUE),  
            mean.O3 = mean(Ozone, na.rm = TRUE),  
            .groups = "drop") %>%  
  drop_na(mean.O3)
```

#13 Finding the dimensions of my monthly summary dataframe
`dim(EPAair.O3.PM25.NC2425.MonthlySummary)`

```
## [1] 238  5
```

#New dimensions include 238 observations of 5 variables. I'm comparing this finding using #drop_na with na.omit by creating a second monthly summary dataframe using that argument.

```
EPAair.O3.PM25.NC2425.MonthlySummary.Omit <-  
  EPAair.O3.PM25.2024.2025.DailySiteLevel.wide %>%  
  group_by(Local.Site.Name, Month, Year) %>%  
  summarize(mean.PM2.5 = mean(PM2.5, na.rm = TRUE),  
            mean.O3 = mean(Ozone, na.rm = TRUE),  
            .groups = "drop") %>%  
  na.omit(mean.O3)
```

#alternative dataframe is created, and I'm checking the dimensions using dim.

```
dim(EPAair.O3.PM25.NC2425.MonthlySummary.Omit)
```

```
## [1] 234  5
```

#The dimensions for this one are 234 observations of 5 variables.

Question 14

14. Why is `drop_na(mean_ozone)` more appropriate here than `na.omit()`? Test both approaches and compare the dimensions of the resulting summary data frames.

Answer: `drop_na` resulted in 238 rows while `na.omit` only resulted in 232 rows. This indicates that `na.omit` was less targeted and deleted six rows that I needed.